	**Carátula para entrega de prácticas**

Laboratorio de computación salas A y B

Profesor:	René Adrián Dávila Pérez	
Asignatura:	Programación Orientada a Objetos	
Grupo:	7	
No de Práctica(s):	11	
Integrante(s):	120002850	323064039
	323071378	323214636
	426059576	426058744
Semestre:	2027-1	
Fecha de entrega:	28-Octubre-2026	
Obervaciones:		

CALIFICACIÓN:

_____

Índice

1. Introducción **3**
- 1.1. Planteamiento del problema 3
- 1.2. Motivación 3
- 1.3. Objetivos 3

2. Marco Teórico **3**

3. Desarrollo **5**
- 3.1. Práctica 5: Encapsulamiento y Manipulación de Objetos 5
- 3.2. Práctica 7: Jerarquía de Herencia de Cuadriláteros 8
- 3.3. Práctica 8: Interfaces y Polimorfismo de Emisiones 11
- 3.4. Proyecto Fase 2: Arquitectura del Sistema Hospitalario 13

4. Resultados **16**

5. Conclusiones **16**

1. Introducción

1.1. Planteamiento del problema

El diseño de sistemas en la programación orientada a objetos demanda herramientas formales que permitan representar con precisión tanto la estructura interna de las clases como su comportamiento en tiempo de ejecución. En aplicaciones con arquitecturas modulares, jerarquías de herencia y contratos mediante interfaces, la ausencia de especificaciones visuales normalizadas dificulta el análisis de responsabilidades, la trazabilidad de interacciones y la verificación de requerimientos. Por ello, resulta indispensable elaborar modelos rigurosos que reflejen fielmente el código desarrollado en las etapas previas del curso, abarcando encapsulamiento, taxonomías polimórficas y flujos de coordinación hospitalaria.

1.2. Motivación

El Lenguaje Unificado de Modelado (UML) constituye el estándar internacional indiscutible en la ingeniería de software para abstraer, documentar y comunicar sistemas de cómputo. Desarrollar diagramas de clases y de secuencia fidedignos no solo afianza la comprensión teórica de conceptos cardinales como multiplicidad, ligadura dinámica y composición, sino que proporciona un mapa técnico indispensable para la auditoría y depuración de software. La consolidación de estos esquemas visuales permite garantizar la cohesión modular y clarificar el ciclo de vida de los objetos en escenarios de ejecución real.

1.3. Objetivos

El objetivo primordial de esta práctica consiste en elaborar diagramas UML normalizados de relaciones de clases y de interacción de secuencia para los proyectos desarrollados a lo largo del curso, excluyendo la práctica seis conforme a las indicaciones académicas. Específicamente, se modelan los sistemas correspondientes a la práctica cinco (encapsulamiento y tipos básicos), práctica siete (jerarquía de cuadriláteros), práctica ocho (interfaces y polimorfismo) y la segunda fase del proyecto integral (gestión hospitalaria), comprobando que cada elemento estructural y mensaje de control guarde total fidelidad con el código fuente.

2. Marco Teórico

El estándar UML [1] define diagramas clasificados en dos grandes categorías: estructurales y de comportamiento. Dentro de los diagramas estructurales, el diagrama de clases modela el catálogo estático del sistema descomponiendo cada entidad en tres compartimentos normalizados: nombre del tipo, atributos con visibilidad y operaciones con parámetros y tipos de retorno [2]. Las visibilidades se denotan mediante símbolos estándar: público (+), privado (−), protegido (#) y paquete (∼). Asimismo, las relaciones entre clases capturan vínculos semánticos específicos: la asociación modela

navegación bidireccional o unidireccional con multiplicidades en ambos extremos; la agregación y composición denotan relaciones de todo y parte, diferenciándose en que la composición implica pertenencia existencial exclusiva representada mediante un rombo relleno en el extremo del contenedor; la generalización expresa herencia mediante flecha con punta triangular cerrada; y la realización manifiesta la implementación de interfaces [3].

Por su parte, los diagramas de interacción, y en particular los diagramas de secuencia, pertenecen a los diagramas de comportamiento y modelan el intercambio temporal de mensajes entre participantes [4]. El tiempo avanza de arriba hacia abajo a lo largo de las líneas de vida verticales discontinuas asociadas a cada objeto o actor. Los rectángulos de activación o focos de control indican los períodos en que un objeto ejecuta un método. Los mensajes síncronos se grafican mediante flechas horizontales con punta triangular rellena hacia el receptor, las respuestas o valores devueltos mediante flechas discontinuas con punta abierta hacia el emisor, y los mensajes de instanciación con el estereotipo de constructor dirigidos hacia la cabecera del nuevo objeto ubicado a la altura de su creación.

3. Desarrollo

3.1. Práctica 5: Encapsulamiento y Manipulación de Objetos

En la práctica cinco se abordó el diseño elemental de clases mediante encapsulamiento estricto en las clases `Fecha` y `Empleado`. La figura 3.1 expone el diagrama de clases correspondiente, donde se aprecian atributos privados y métodos selectores y modificadores debidamente tipados. La clase `MainEmpleado` actúa como cliente que instancia dos objetos de tipo `Empleado` para calcular el salario anual y aplicar el incremento salarial establecido. La figura 3.2 modela la secuencia de ejecución de este caso representativo, exhibiendo la llamada inicial de consola, la construcción sucesiva de ambas instancias a su altura respectiva y las consultas y modificaciones síncronas.

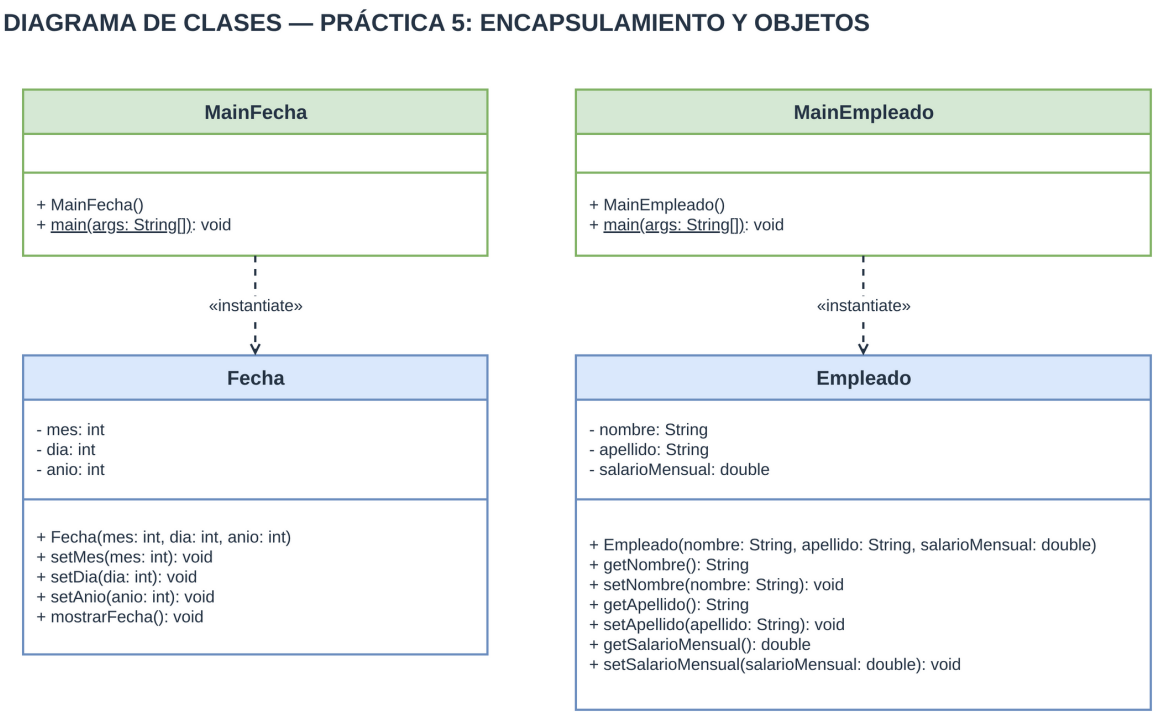


Figura 3.1: Diagrama de clases UML de la práctica 5 (Fecha y Empleado).

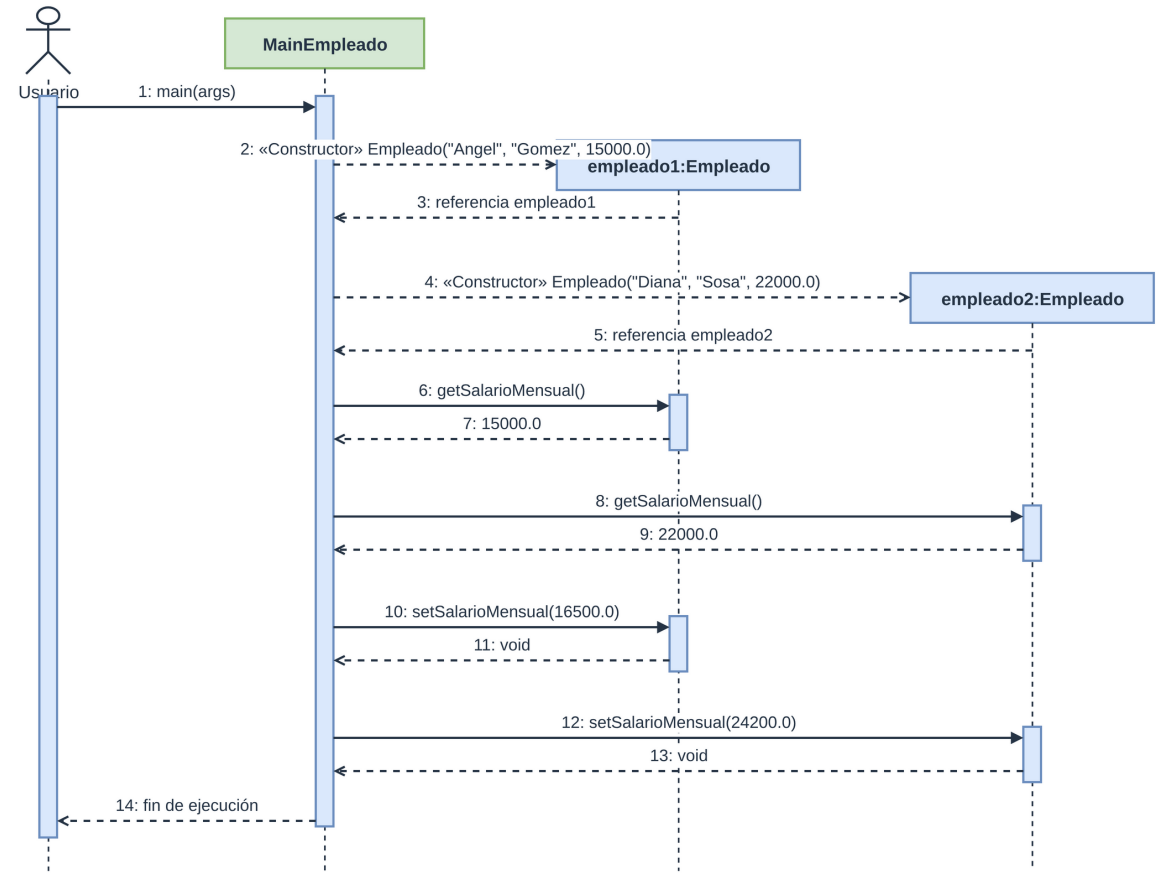


Figura 3.2: Diagrama de secuencia UML de la práctica 5 (Ejecución de MainEmpleado).

3.2. Práctica 7: Jerarquía de Herencia de Cuadriláteros

La práctica siete implementó una taxonomía geométrica en la que la superclase `Cuadrilatero` almacena cuatro vértices bidimensionales modelados mediante la clase `Punto`. En la figura 3.3 se ilustra la relación de composición fuerte entre `Cuadrilatero` y `Punto` con multiplicidad obligatoria de 1 a 4. Posteriormente, se despliega la cadena de generalización: `Trapezoide` hereda de `Cuadrilatero`, `Paralelogramo` extiende a `Trapezoide`, `Rectangulo` especializa a `Paralelogramo` y finalmente `Cuadrado` deriva de `Rectangulo`. La figura 3.4 documenta el diagrama de secuencia donde la clase `Main` instancia los cuatro vértices requeridos, crea el objeto `Cuadrado` y solicita el cálculo de área, el cual realiza una delegación anidada sobre `getLado()` y `getLongitud()`.

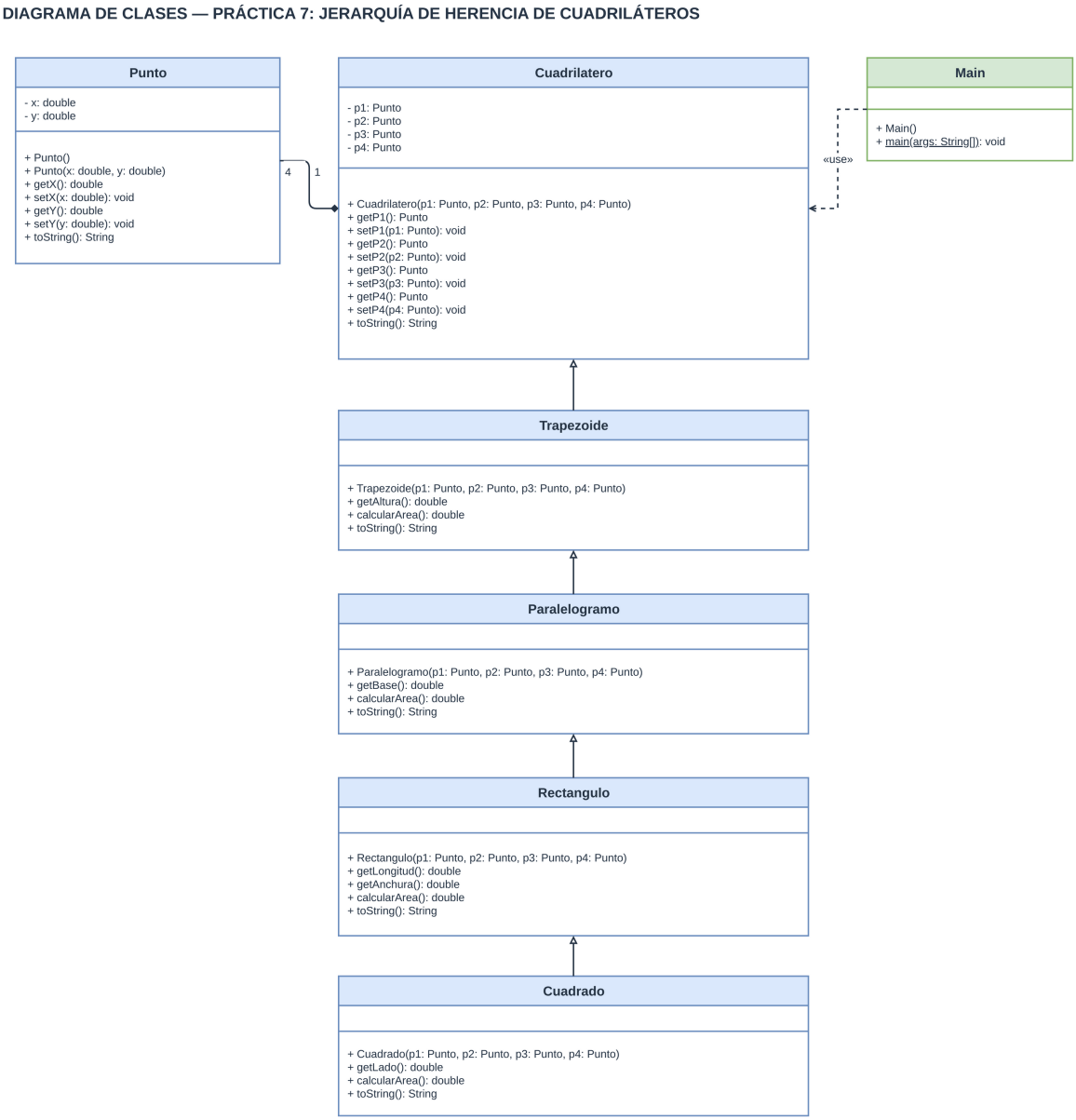


Figura 3.3: Diagrama de clases UML de la práctica 7 (Jerarquía de Cuadriláteros).

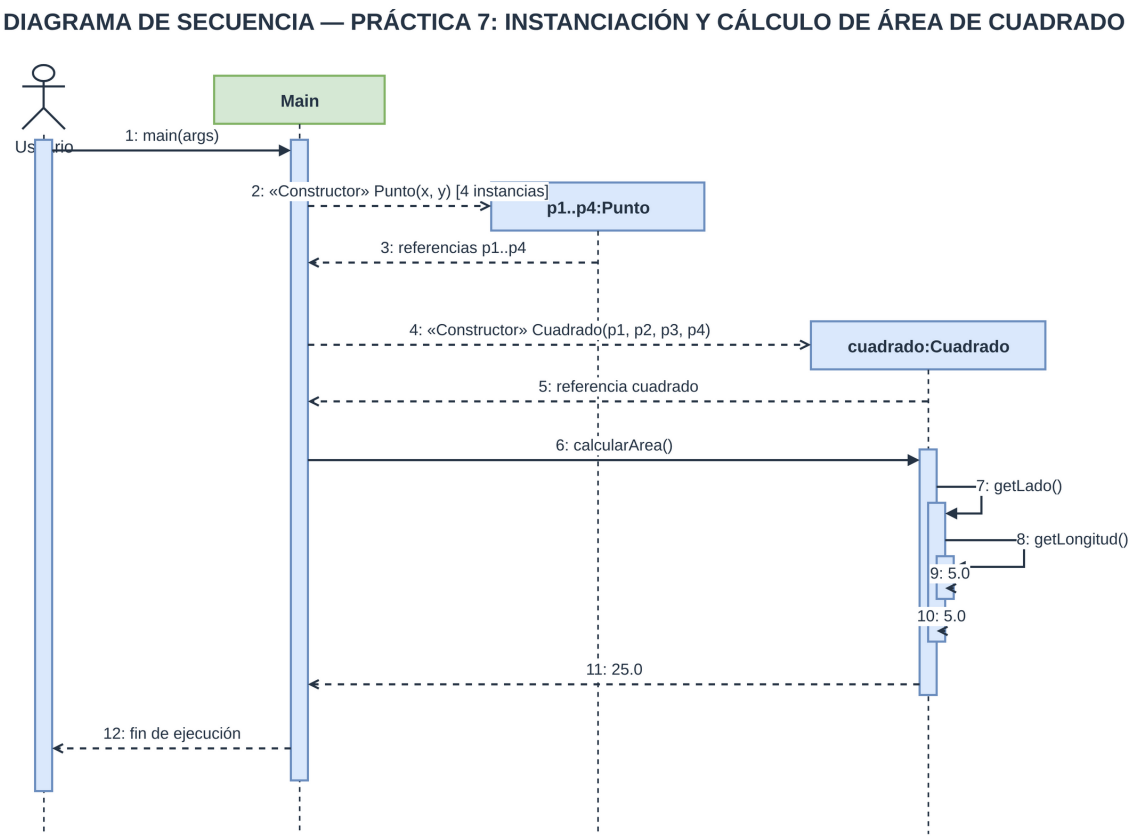


Figura 3.4: Diagrama de secuencia UML de la práctica 7 (Instanciación y cálculo de área de Cuadrado).

3.3. Práctica 8: Interfaces y Polimorfismo de Emisiones

En la práctica ocho se exploró el desacoplamiento estructurado implementando la interfaz común `HuellaCarbono`, la cual define la firma polimórfica `getHuellaCarbono()`. Como se exhibe en la figura 3.5, las entidades `Edificio`, `Auto` y `Bicicleta` implementan este contrato formal. La figura 3.6 presenta el diagrama de secuencia donde se observa la construcción de los objetos y su agrupamiento en una colección homogénea, sobre la cual un fragmento combinado de ciclo (`loop`) efectúa llamadas polimórficas que resuelven dinámicamente el valor de emisiones propio de cada tipo de entidad.

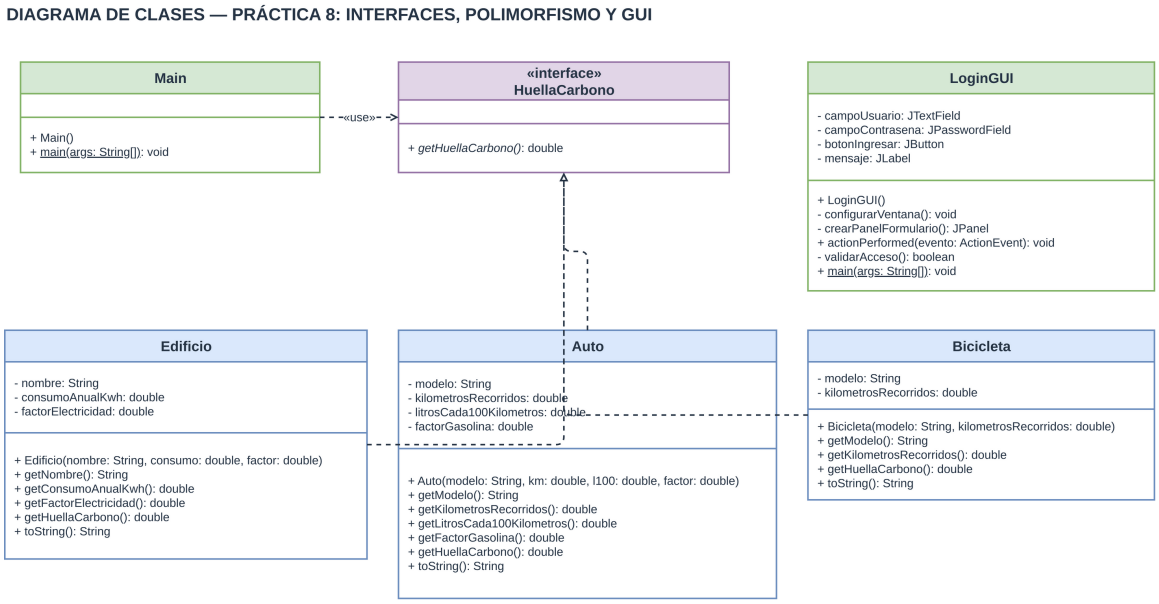


Figura 3.5: Diagrama de clases UML de la práctica 8 (Huella de carbono y ventana GUI).

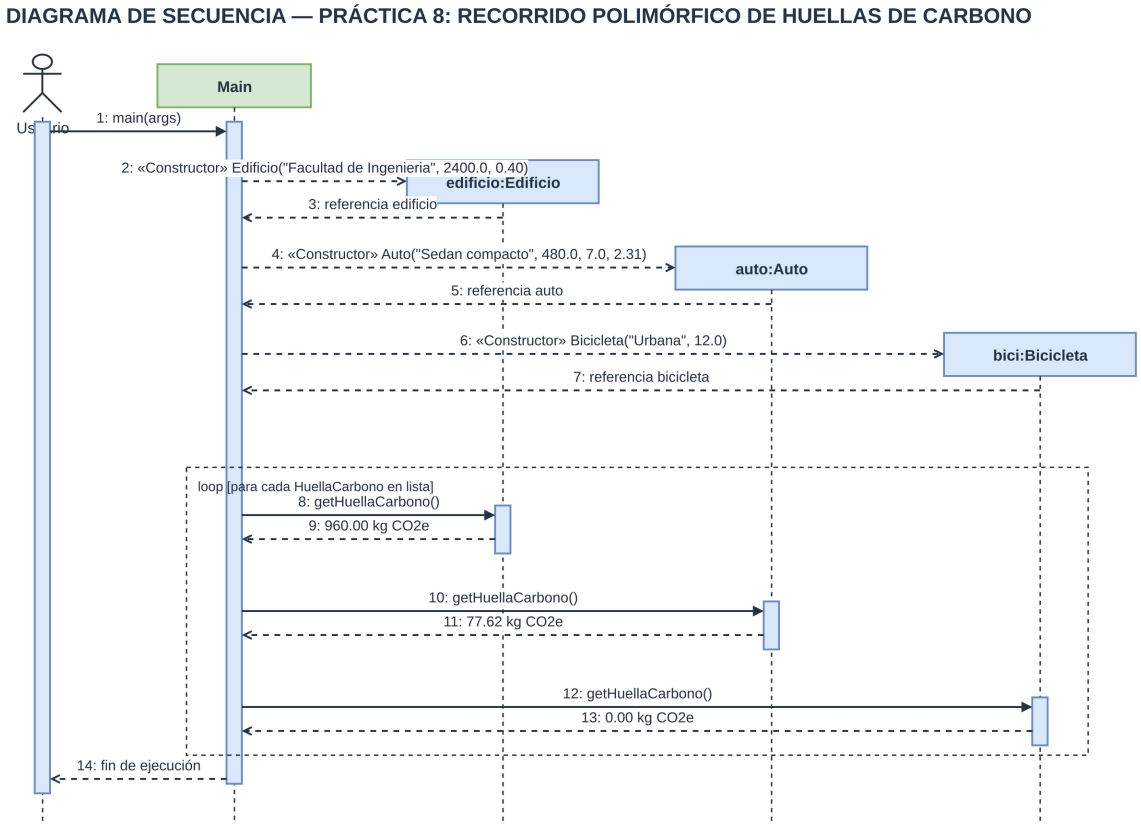


Figura 3.6: Diagrama de secuencia UML de la práctica 8 (Recorrido polimórfico de emisiones).

3.4. Proyecto Fase 2: Arquitectura del Sistema Hospitalario

La segunda fase del proyecto integral representó el reto arquitectónico más complejo, integrando roles clínicos y administrativos alrededor de la clase `Sistema`. La figura 3.7 detalla la estructura global: la interfaz `AtencionMedica` es implementada por las clases abstractas `Medico` y `Enfermero`; la jerarquía médica se subdivide en `MedicoGeneral` y `MedicoEspecialista`, mientras que el personal de enfermería comprende `EnfermeroEspecialista` y el supervisor `JefeEnfermeria`. Los pacientes se modelan bajo la abstracción `Paciente`, extendida en `PacienteGeneral` y `PacienteEspecialidad`. Las relaciones de agregación y asociación plasman restricciones operativas exactas: el hospital coordina múltiples médicos y enfermeros; cada médico administra hasta diez pacientes, y cada enfermero asiste un máximo de tres pacientes.

En la figura 3.8 se sintetiza el diagrama de secuencia para un flujo de atención médica integral. El escenario abarca el registro de la persona en el catálogo del hospital, la solicitud y atención de la consulta médica, la prescripción y tratamiento inicial, la autorización de medicamentos controlados por el jefe de área y la aplicación final por el enfermero especialista.

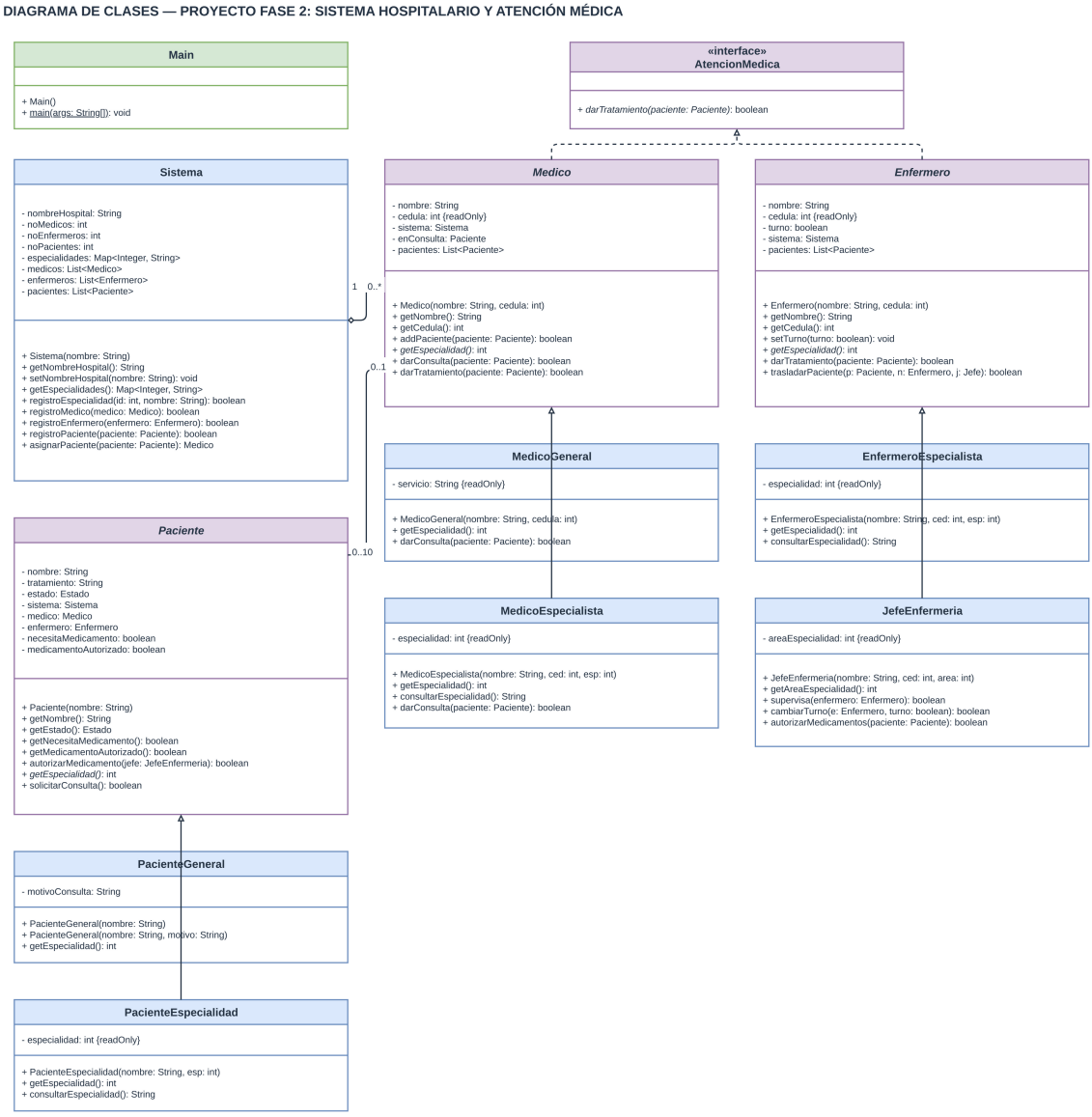


Figura 3.7: Diagrama de clases UML del Proyecto Fase 2 (Sistema Hospitalario).

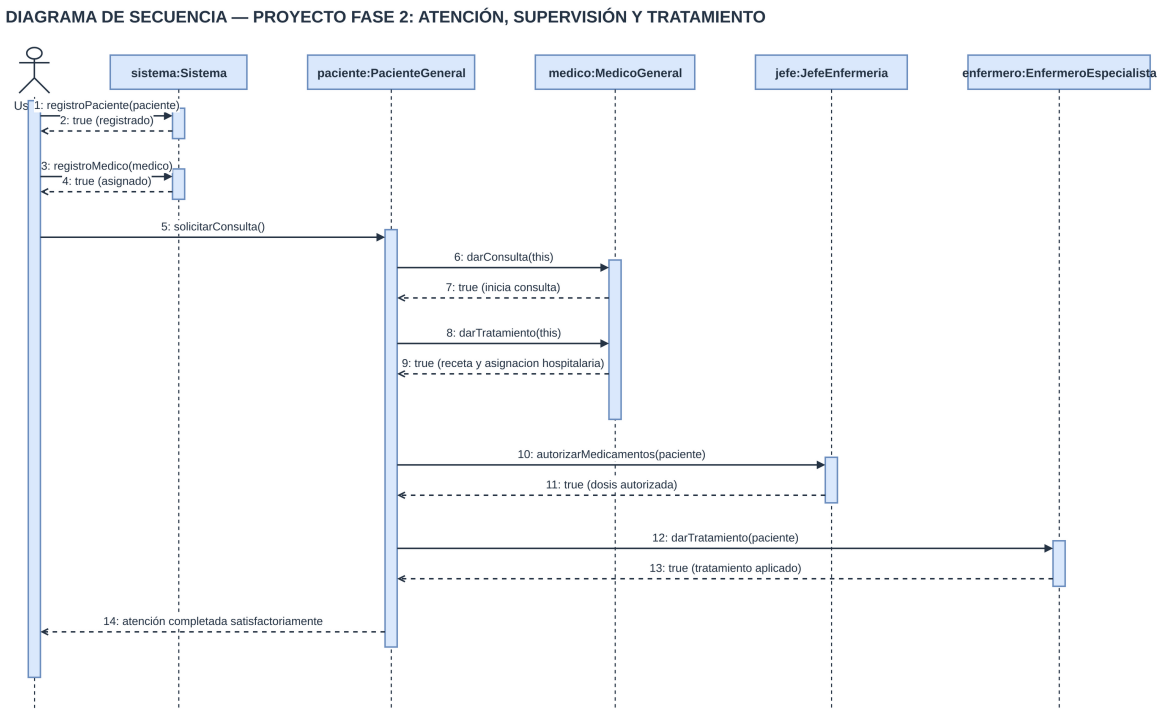


Figura 3.8: Diagrama de secuencia UML del Proyecto Fase 2 (Flujo de atención y supervisión).

4. Resultados

La formulación de los ocho diagramas UML permitió constatar la total congruencia entre el diseño conceptual y las implementaciones de Java previamente estabilizadas. En la tabla 1 se resume la cobertura alcanzada por cada esquema respecto a clases, miembros y participantes dinámicos.

Tabla 1: Inventario técnico de los diagramas UML elaborados.

Sistema	**Diagramas**	**Categoría**	**Elementos clave**
Práctica 5	Clases y Secuencia	Encapsulamiento	4 clases; llamadas y retornos síncronos
Práctica 7	Clases y Secuencia	Herencia	Composición 1:4; 5 niveles de herencia
Práctica 8	Clases y Secuencia	Interfaces	1 interfaz; 3 realizaciones; ciclo loop
Proyecto 2	Clases y Secuencia	Sistema integral	12 clases; roles clínicos y supervisión

Cada una de las figuras integradas en el documento cumple estrictamente con el estándar educativo acordado: geometría ortogonal sin curvaturas, tipografía proporcional legible superior a 10 puntos en impresión y anclaje exacto de extremos de conexión. Los diagramas de secuencia evidencian la temporalidad rigurosa del flujo de ejecución sin recurrir a invocaciones artificiales ni cruces improcedentes de líneas de vida.

5. Conclusiones

El desarrollo sistemático de diagramas UML de clases y secuencia para los sistemas implementados en el curso demostró la importancia crítica de contar con una representación gráfica formal antes y durante la codificación. Mediante los diagramas de clases fue posible validar que las responsabilidades estuvieran correctamente distribuidas, que los niveles de visibilidad protegieran el estado interno de los objetos y que las multiplicidades reflejaran con precisión las reglas de negocio del problema.

Asimismo, los diagramas de secuencia permitieron contrastar la dinámica real de ejecución de cada programa con el flujo previsto, evidenciando cómo el polimorfismo y la delegación modular simplifican la arquitectura de control. En particular, la modelación de la fase dos del proyecto hospitalario exhibió cómo un esquema de interacción bien estructurado previene ambigüedades en la coordinación de múltiples roles y en la asignación de recursos limitados. En conjunto, estos entregables consolidan una metodología de ingeniería de software robusta, legible y académicamente verificable.

Referencias

[1] Object Management Group. *Unified Modeling Language Specification (OMG UML), Version 2.5.1.* OMG Document Formal/2017-12-05, Needham, MA, 2017.

[2] G. Booch, J. Rumbaugh y I. Jacobson. *The Unified Modeling Language User Guide.* 2.ª ed., Addison-Wesley Professional, Boston, MA, 2005.

[3] M. Fowler. *UML Distilled: A Brief Guide to the Standard Object Modeling Language.* 3.ª ed., Addison-Wesley, Boston, MA, 2004.

[4] R. S. Pressman y B. R. Maxim. *Software Engineering: A Practitioner's Approach.* 9.ª ed., McGraw-Hill Education, Nueva York, NY, 2020.